📬 DM (Direct Message) 기능 완전 분석

✅ 현재 상태: DM 기능이 완전히 구현되어 있습니다!

이전 분석과 달리, DM 기능은 완전히 구현되어 있으며 실제로 작동 중입니다.

🗄️ 1. 데이터베이스 구조 (Firestore)

1.1 Conversations 컬렉션

```
conversations/{conversationId}
 ├─── participants: [userId1, userId2]
 ├─── participantNames: {userId: nickname}
 ├─── participantPhotos: {userId: photoURL}
 ├─── isAnonymous: {userId: boolean}
 ├─── lastMessage: string
 ├─── lastMessageTime: timestamp
 ├─── lastMessageSenderId: string
```

├── unreadCount: {userId: number}

├── postId: string? (익명 게시글 DM인 경우)

├── dmTitle: string? (게시글 제목)

├── createdAt: timestamp

├── updatedAt: timestamp

├── archivedBy: [userId] (보관한 사용자 목록)

├── userLeftAt: {userId: timestamp?} (나간 시간 기록)

└── rejoinedAt: {userId: timestamp?} (재진입 시간 기록)

1.2 Messages 서브컬렉션

conversations/{conversationId}/messages/{messageId}

├── senderId: string

├── text: string (최대 500자)

├── createdAt: timestamp

```
├── isRead: boolean
└── readAt: timestamp?
```

1.3 ConversationId 생성 규칙

- 일반 DM: {작은uid}_{큰uid} (사전순 정렬)
- 익명 게시글 DM: anon_{작은uid}_{큰uid}_{postId}
- 재진입 시 새 방: {기존id}__{timestamp}

🔐 2. Firestore Security Rules

firestore.rulesLines 582-679

```
// ===== DM(Direct Message) 컬렉션 =====
match /conversations/{conversationId} {
  // conversationId 형식: {작은uid}_{큰uid} (사전순 정렬)

  // 개별 문서 읽기: 로그인한 사용자 모두 허용
```

```
  allow get: if request.auth != null;

  // 목록 조회: 로그인한 사용자 모두 허용
  allow list: if request.auth != null;

  // 생성: 로그인한 사용자 모두 허용 (명시적으로)
  allow create: if request.auth != null;

  // 쓰기 권한: 로그인한 사용자 모두 허용 (create, update, delete 포함)
  allow write: if request.auth != null;

  // 수정: (1) 메시지 관련 필드/보관, (2) '나가기' 허용
  allow update: if request.auth != null &&
```

```
    request.auth.uid in resource.data.participants &&
    (
      (
        // 일반 메시지 관련 변경 + 보관
        request.resource.data.participants == resource.data.participants &&
        request.resource.data.diff(resource.data).affectedKeys()
          .hasOnly(['lastMessage', 'lastMessageTime', 'lastMessageSenderId', 'unreadCount', 'updatedAt', 'archivedBy'])
      ) ||
      (
        // '나가기': participants에서 본인만 제거 (N → N-1), 기타 필드는 unreadCount/updatedAt만 변경 가능
```

```
        request.auth.uid in resource.data.participants &&
        !(request.auth.uid in request.resource.data.participants) &&
        request.resource.data.participants.size() == resource.data.participants.size() - 1 &&
        // 남아있는 모든 참여자는 기존 참여자 중 일부여야 함 (여기서는 1명만 남는 구조)
        request.resource.data.participants.size() == 1 &&
        request.resource.data.participants[0] in resource.data.participants &&
        request.resource.data.diff(resource.data).affectedKeys()
          .hasOnly(['participants','unreadCount','updatedAt'])
      )
```

```
    );

  // 삭제: 대화방 참여자라면 삭제 가능 (클라이언트에서 메시지 먼저 삭제 후 문서 삭제)
  allow delete: if request.auth != null && request.auth.uid in resource.data.participants;

  // ===== 메시지 서브컬렉션 =====
  match /messages/{messageId} {
    // 읽기: 대화방 참여자만
    // get() 호출 최소화를 위해 함수 사용
    function isParticipant() {
      // 대화방이 존재하면 participants 체크
      return exists(/databases/$(database)/documents/conversations/$(conversationId)) &&
             request.auth.uid in get(/databases/$(database)/documents/conversations/$(convers
```

```
ationId)).data.participants;
    }

    // conversationId에서 사용자 ID 추출 (일반: uid1_uid2, 익명: anon_uid1_uid2_postId)
    function isMyConversation() {
      let parts = conversationId.split('_');
      let isAnonymous = conversationId.matches('anon_.*');

      // 익명 대화방: anon_uid1_uid2_postId 형식
      // parts[0] = 'anon', parts[1] = uid1, parts[2] = uid2, parts[3+] = postId
      return isAnonymous
        ? (parts.size() >= 3 && (request.auth.uid == parts[1] || request.auth.uid == parts[2]))
```

```
        : (request.auth.uid in parts);
    }

    // 개별 메시지 읽기
    allow get: if request.auth != null && (isPart
icipant() || isMyConversation());

    // 메시지 목록 조회
    allow list: if request.auth != null && (isParti
cipant() || isMyConversation());

    // 생성: 핵심 검증만 유지
    allow create: if request.auth != null &&
      (isParticipant() || isMyConversation()) &&
      // 발신자는 본인
      request.auth.uid == request.resource.data.
```

```
senderId &&
      // 메시지는 비어있지 않음
      request.resource.data.text.size() > 0 &&
      // 메시지 길이 제한 (500자)
      request.resource.data.text.size() <= 500 &
&
      // 초기 읽음 상태는 false
      request.resource.data.isRead == false;

    // 수정: 읽음 처리 로직만 유지
    allow update: if request.auth != null &&
      (isParticipant() || isMyConversation()) &&
      // 수신자만 읽음 처리 가능
      request.auth.uid != resource.data.senderId
 &&
      // isRead만 false→true로 변경
```

```
        resource.data.isRead == false &&
        request.resource.data.isRead == true &&
        // 다른 필드는 변경 불가
        request.resource.data.diff(resource.data).affectedKeys()
          .hasOnly(['isRead', 'readAt']);

    // 삭제: 대화방 참여자는 자신의 채팅방 하위 메시지를 삭제 가능 (대화방 완전 삭제 시 사용)
    allow delete: if request.auth != null && (isParticipant() || isMyConversation());
  }
}
```

🧮 3. 모델 클래스

3.1 DMMessage 모델

dm_message.dartLines 1-80

```
// lib/models/dm_message.dart
// DM 메시지 데이터 모델
// Firestore conversations/{id}/messages 서브컬렉션에 대응
import 'package:cloud_firestore/cloud_firestore.dart';
class DMMessage {
  final String id;
  final String senderId;
  final String text;
  final DateTime createdAt;
  final bool isRead;
  final DateTime? readAt;
  DMMessage({
    required this.id,
```

```
    required this.senderId,
    required this.text,
    required this.createdAt,
    required this.isRead,
    this.readAt,
  });
  /// Firestore 문서에서 DMMessage 객체 생성
  factory DMMessage.fromFirestore(Document
Snapshot doc) {
    final data = doc.data() as Map<String, dyna
mic>;
    return DMMessage(
      id: doc.id,
      senderId: data['senderId'] ?? '',
      text: data['text'] ?? '',
      createdAt: (data['createdAt'] as Timestam
```

```
p).toDate(),
      isRead: data['isRead'] ?? false,
      readAt: data['readAt'] != null
          ? (data['readAt'] as Timestamp).toDate(
)
          : null,
    );
  }
  /// DMMessage 객체를 Firestore 문서로 변환
  Map<String, dynamic> toFirestore() {
    return {
      'senderId': senderId,
      'text': text,
      'createdAt': Timestamp.fromDate(created
At),
      'isRead': isRead,
```

```
      if (readAt != null) 'readAt': Timestamp.fromDate(readAt!),
    };
  }
  /// 메시지가 현재 사용자가 보낸 것인지 확인
  bool isMine(String currentUserId) {
    return senderId == currentUserId;
  }
  /// 메시지 복사 (읽음 상태 업데이트용)
  DMMessage copyWith({
    String? id,
    String? senderId,
    String? text,
    DateTime? createdAt,
    bool? isRead,
    DateTime? readAt,
```

```
  }) {
    return DMMessage(
      id: id ?? this.id,
      senderId: senderId ?? this.senderId,
      text: text ?? this.text,
      createdAt: createdAt ?? this.createdAt,
      isRead: isRead ?? this.isRead,
      readAt: readAt ?? this.readAt,
    );
  }
  @override
  String toString() {
    return 'DMMessage(id: $id, senderId: $send
erId, text: ${text.length > 20 ? '${text.substrin
g(0, 20)}...' : text}, isRead: $isRead)';
  }
```

}

3.2 Conversation 모델

conversation.dartLines 1-220

```
// lib/models/conversation.dart
// DM 대화방 데이터 모델
// Firestore conversations 컬렉션에 대응
import 'package:cloud_firestore/cloud_firestore.dart';
import 'dart:math' as math;
class Conversation {
  final String id;
  final List<String> participants;
  final Map<String, String> participantNames;
  final Map<String, String> participantPhotos;
  final Map<String, bool> isAnonymous;
  final String lastMessage;
```

```
  final DateTime lastMessageTime;
  final String lastMessageSenderId;
  final Map<String, int> unreadCount;
  final String? postId;
  // 익명 게시글로부터 시작된 DM의 경우 채팅방 표시용 제목(게시글 제목)
  final String? dmTitle;
  final DateTime createdAt;
  final DateTime updatedAt;
  final List<String> archivedBy; // 이 대화방을 보관(삭제)한 사용자 uid 목록
  final Map<String, DateTime?> userLeftAt; // 각 사용자가 언제 나갔는지 기록
  final Map<String, DateTime?> rejoinedAt; // 각 사용자가 언제 다시 들어왔는지 기록
  Conversation({
    required this.id,
```

```
  required this.participants,
  required this.participantNames,
  required this.participantPhotos,
  required this.isAnonymous,
  required this.lastMessage,
  required this.lastMessageTime,
  required this.lastMessageSenderId,
  required this.unreadCount,
  this.postId,
  this.dmTitle,
  required this.createdAt,
  required this.updatedAt,
  this.archivedBy = const [],
  this.userLeftAt = const {},
  this.rejoinedAt = const {},
});
```

```
  /// Firestore 문서에서 Conversation 객체 생성
  factory Conversation.fromFirestore(DocumentSnapshot doc) {
    final data = doc.data() as Map<String, dynamic>;
    // userLeftAt 필드 파싱
    final userLeftAtData = data['userLeftAt'] as Map<String, dynamic>? ?? {};
    final userLeftAt = <String, DateTime?>{};
    for (final entry in userLeftAtData.entries) {
      userLeftAt[entry.key] = entry.value != null

          ? (entry.value as Timestamp).toDate()
          : null;
    }
    // rejoinedAt 필드 파싱
    final rejoinedAtData = data['rejoinedAt'] as
```

```
Map<String, dynamic>? ?? {};
    final rejoinedAt = <String, DateTime?>{};
    for (final entry in rejoinedAtData.entries) {
      rejoinedAt[entry.key] = entry.value != null


          ? (entry.value as Timestamp).toDate()
          : null;
    }
    return Conversation(
      id: doc.id,
      participants: List<String>.from(data['parti
cipants'] ?? []),
      participantNames: Map<String, String>.fr
om(data['participantNames'] ?? {}),
      participantPhotos: Map<String, String>.fr
om(data['participantPhotos'] ?? {}),
      isAnonymous: Map<String, bool>.from(da
```

```
ta['isAnonymous'] ?? {}),
      lastMessage: data['lastMessage'] ?? '',
      lastMessageTime: (data['lastMessageTime'
] as Timestamp).toDate(),
      lastMessageSenderId: data['lastMessageSe
nderId'] ?? '',
      unreadCount: Map<String, int>.from(data
['unreadCount'] ?? {}),
      postId: data['postId'],
      dmTitle: data['dmTitle'],
      createdAt: (data['createdAt'] as Timestam
p).toDate(),
      updatedAt: (data['updatedAt'] as Timesta
mp).toDate(),
      archivedBy: (data['archivedBy'] as List?)?.
map((e) => e.toString()).toList() ?? const [],
      userLeftAt: userLeftAt,
```

```
      rejoinedAt: rejoinedAt,
    );
  }
  /// 상대방 사용자 ID 가져오기
  String getOtherUserId(String currentUserId) {
    return participants.firstWhere(
      (id) => id != currentUserId,
      orElse: () => participants[0],
    );
  }
  /// 상대방 이름 가져오기 (익명 처리 포함)
  String getOtherUserName(String currentUserI
d) {
    final otherUserId = getOtherUserId(current
UserId);
```

```
    // 상대방이 익명인 경우
    if (isAnonymous[otherUserId] == true) {
      return '익명'; // 익명 표시 (UI에서 로컬라이제이션 처리)
    }

    return participantNames[otherUserId] ?? 'Unknown';
  }
  /// 상대방이 익명인지 확인
  bool isOtherUserAnonymous(String currentUserId) {
    final otherUserId = getOtherUserId(currentUserId);
    return isAnonymous[otherUserId] ?? false;
  }
  /// 내 읽지 않은 메시지 수 가져오기
```

```
  int getMyUnreadCount(String currentUserId)
{
    return unreadCount[currentUserId] ?? 0;
  }
}
```

🔧 4. 서비스 레이어 (DMService)

핵심 기능:

1. 대화방 생성/조회: getOrCreateConversation()
2. 메시지 전송: sendMessage()
3. 메시지 스트림: getMessages()
4. 읽음 처리: markAsRead()
5. 대화방 나가기: leaveConversation() (인스타그램 방식)
6. 총 읽지 않은 메시지 수: getTotalUnreadCount()

dm_service.dartLines 13-1449

```
class DMService {
  final FirebaseFirestore _firestore = FirebaseFirestore.instance;
  final FirebaseAuth _auth = FirebaseAuth.instance;
  final NotificationService _notificationService = NotificationService();
  /// conversationId 생성 (사전순 정렬)
  String _generateConversationId(String uid1, String uid2, {bool anonymous = false, String? postId}) {
    final sorted = [uid1, uid2]..sort();


    if (!anonymous) {
      return '${sorted[0]}_${sorted[1]}';
    }
```

```
    final suffix = (postId != null && postId.isNotEmpty) ? postId : DateTime.now().millisecondsSinceEpoch.toString();
    return 'anon_${sorted[0]}_${sorted[1]}_$suffix';
  }
  /// 대화방 가져오기 또는 생성
  Future<String?> getOrCreateConversation(
    String otherUserId, {
    String? postId,
    bool isOtherUserAnonymous = false,
    bool isFriend = false,
  }) async {
    // ... 구현 생략 (위 코드 참조)
  }
  /// 메시지 전송
```

```
  Future<bool> sendMessage(String conversationId, String text) async {
    // ... 구현 생략 (위 코드 참조)
  }
  /// 메시지 목록 스트림
  Stream<List<DMMessage>> getMessages(String conversationId, {int limit = 50, DateTime? visibilityStartTime}) {
    // ... 구현 생략 (위 코드 참조)
  }
  /// 읽음 처리
  Future<void> markAsRead(String conversationId) async {
    // ... 구현 생략 (위 코드 참조)
  }
  /// 대화방 나가기 (인스타그램 방식)
  Future<void> leaveConversation(String conv
```

```
ersationId) async {
    // ... 구현 생략 (위 코드 참조)
  }
  /// 총 읽지 않은 메시지 수 스트림
  Stream<int> getTotalUnreadCount() {
    // ... 구현 생략 (위 코드 참조)
  }
}
```

🖥️ 5. UI 화면

5.1 DM 목록 화면 (DMListScreen)

- 위치: lib/screens/dm_list_screen.dart
- 기능:
- 친구/익명 탭 필터
- 대화방 목록 표시 (실시간 업데이트)
- 읽지 않은 메시지 배지

- 친구 선택하여 새 대화 시작
- 대화방 클릭 시 채팅 화면으로 이동

5.2 DM 채팅 화면 (DMChatScreen)

- 위치: lib/screens/dm_chat_screen.dart
- 기능:
- 메시지 목록 표시 (역순, 최신 메시지가 아래)
- 메시지 전송
- 읽음 표시
- 익명 게시글 DM인 경우 게시글로 돌아가기 배너
- 대화방 나가기
- 차단 기능 (준비 중)

🔗 6. 통합 지점

6.1 메인 네비게이션

main_screen.dartLines 107-114

```
List<Widget> get _screens => [
  BoardScreen(searchQuery: _searchController.text),
  // 알림에서 온 모임은 최초 1회만 자동 오픈되도록 전달
  MeetupHomePage(initialMeetupId: _pendingMeetupId),
  const DMListScreen(),   // ← DM 목록 화면
  const MyPageScreen(),
  const FriendsMainPage(),
];
```

6.2 프로필에서 DM 시작

- 위치: lib/widgets/profile_action_buttons.dart
- 프로필 화면에서 "메시지" 버튼 클릭 시 DM 화면으로 이동

6.3 익명 게시글에서 DM 시작

- 익명 게시글 작성자에게 DM 보내기 가능
- anon_{uid1}_{uid2}_{postId} 형식의 대화방 생성

🔔 7. 알림 시스템

7.1 DM 수신 알림

- 알림 타입: dm_received
- 알림 생성: DMService.sendMessage() 내에서 자동 생성
- FCM 푸시 알림: NotificationService를 통해 전송

7.2 읽지 않은 메시지 배지

- 메인 화면: DM 탭에 총 읽지 않은 메시지 수 표시
- 대화방 목록: 각 대화방별 읽지 않은 메시지 수 표시 (실시간 StreamBuilder)
- 실시간 업데이

트: DMService.getTotalUnreadCount() 스트림 사용

🎯 8. 주요 특징

8.1 인스타그램 방식 대화방 나가기

- 나간 사용자는 이전 메시지를 볼 수 없음
- 상대방은 모든 메시지 유지
- userLeftAt 필드로 나간 시간 기록
- 새 메시지 전송 시 다시 대화방에 진입

8.2 익명 DM 지원

- 익명 게시글 작성자와 DM 가능
- 양방향 익명 (서로의 신원 숨김)
- 게시글 제목이 대화방 제목으로 표시

8.3 실시간 배지 업데이트

- 카카오톡처럼 읽음 처리 즉시 배지 사라짐
- StreamBuilder로 실시간 업데이트

- getActualUnreadCountStream() 사용

8.4 차단 확인

- DM 전송 전 차단 여부 확인
- 차단된 사용자에게는 DM 불가

📝 9. Cursor AI에 명령 시 참고사항

9.1 동일한 DM 시스템 구현 시 필요한 정보

데이터베이스 구조:

- Firestore conversations 컬렉션 (위 1번 참조)
- conversations/{id}/messages 서브컬렉션
- ConversationId 생성 규칙 (일반/익명/재진입)

보안 규칙:

- firestore.rules 파일의 582~679번 라인 참조
- 참여자만 읽기/쓰기 가능
- 메시지 길이 제한 (500자)

- 읽음 처리 규칙

모델 클래스:

- DMMessage: 메시지 데이터
- Conversation: 대화방 데이터

서비스:

- DMService: 모든 DM 관련 로직
- NotificationService: 알림 전송

UI:

- DMListScreen: 대화방 목록
- DMChatScreen: 채팅 화면
- 메인 네비게이션에 통합

핵심 기능:

1. 대화방 생성/조회
2. 메시지 전송/수신 (실시간 스트림)
3. 읽음 처리
4. 인스타그램 방식 나가기

5. 익명 DM 지원

6. 실시간 배지 업데이트

9.2 명령 예시

"현재 DM 시스템과 동일한 기능을 가진 DM 시스템을 구현해줘. 다음 사항을 포함해야 해:

- Firestore conversations 컬렉션 구조 (participants, unreadCount, userLeftAt 등)
- 보안 규칙 (참여자만 읽기/쓰기, 메시지 길이 제한)
- DMMessage와 Conversation 모델
- DMService (대화방 생성, 메시지 전송, 읽음 처리, 나가기)
- DMListScreen과 DMChatScreen UI
- 인스타그램 방식 나가기 (userLeftAt 기록)
- 익명 DM 지원 (anon_ prefix)
- 실시간 배지 업데이트 (StreamBuilder)"

이 정보를 바탕으로 Cursor AI에 명령하면 현재 시스템과 동일한 DM 기능을 정확하게 재현할 수 있습니다! 🎉